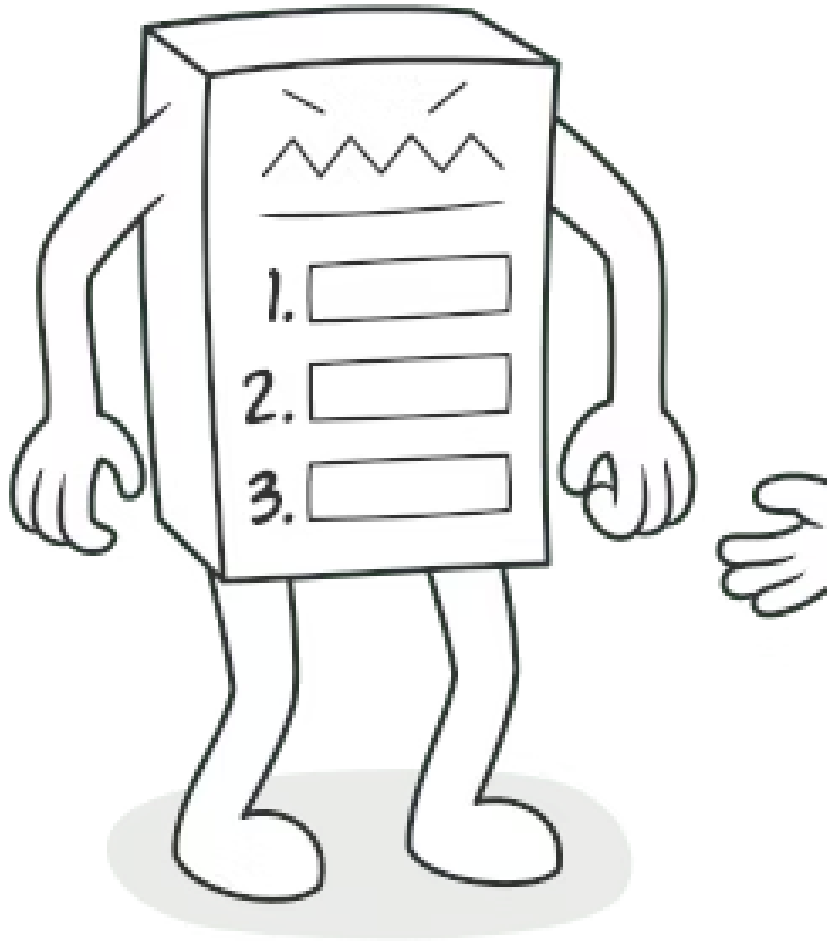




PATRÓN DE DISEÑO DE COMPORTAMIENTO

Template Method

También llamado: **Método Plantilla**

Un patrón que define el esqueleto de un algoritmo en la superclase, permitiendo que las subclases sobrescriban pasos específicos sin alterar la estructura general.

Agenda de la Presentación

01

Propósito

Definición y objetivo del patrón Template Method

03

Solución

Dividir algoritmos en pasos reutilizables

05

Estructura y Pseudocódigo

Diagrama UML y ejemplo con videojuego de estrategia

02

Problema

Código duplicado y condicionales en aplicaciones

04

Analogía

Construcción de viviendas en masa

06

Aplicabilidad e Implementación

Cuándo usar, cómo implementar, pros, contras y relaciones

Propósito

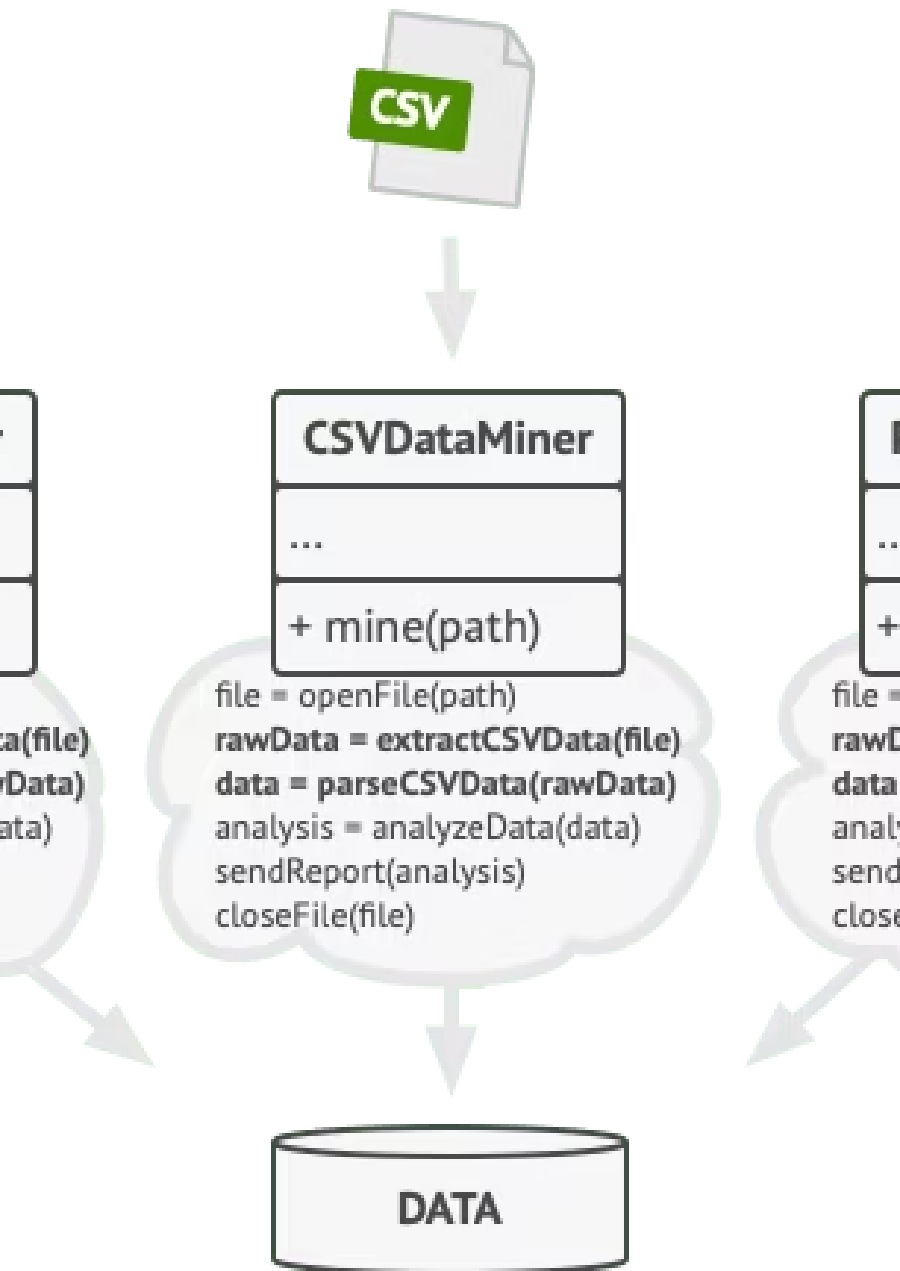
Template Method es un patrón de diseño de comportamiento que define el **esqueleto de un algoritmo** en la superclase pero permite que las subclases sobrescriban pasos del algoritmo **sin cambiar su estructura**.

Esto significa que la lógica general del algoritmo permanece intacta en la clase padre, mientras que los detalles específicos de cada paso pueden ser personalizados por las clases hijas según sus necesidades particulares.

El Problema

Imagina que estás creando una **aplicación de minería de datos** que analiza documentos corporativos. Los usuarios suben documentos en varios formatos (**PDF, DOC, CSV**) y la aplicación intenta extraer la información relevante en un formato uniforme.





Código Duplicado: El Verdadero Problema

Las clases de minería de datos contenían **mucho código duplicado**. Aunque el código para gestionar distintos formatos de datos es totalmente diferente en todas las clases, el código para **procesar y analizar los datos es casi idéntico**.

¿No sería genial deshacerse de la duplicación de código, dejando intacta la estructura del algoritmo?

Problema con el Código Cliente

Hay otro problema relacionado con el **código cliente** que utiliza esas clases:

- Tiene **muchos condicionales** que eligen un curso de acción dependiendo de la clase del objeto de procesamiento
- Si las tres clases de procesamiento tienen una **interfaz común** o una **clase base**, puedes eliminar los condicionales
- Se puede utilizar el **polimorfismo** al invocar métodos en un objeto de procesamiento



Idea Clave

El polimorfismo permite reemplazar condicionales complejos por una estructura limpia y extensible basada en herencia.

La Solución

El patrón Template Method sugiere que **dividas un algoritmo en una serie de pasos**, conviertas estos pasos en métodos y coloques una serie de llamadas a esos métodos dentro de un único **método plantilla**.

Pasos Abstractos

Deben ser implementados por cada subclase

Pasos con Implementación

Cuentan con una implementación por defecto

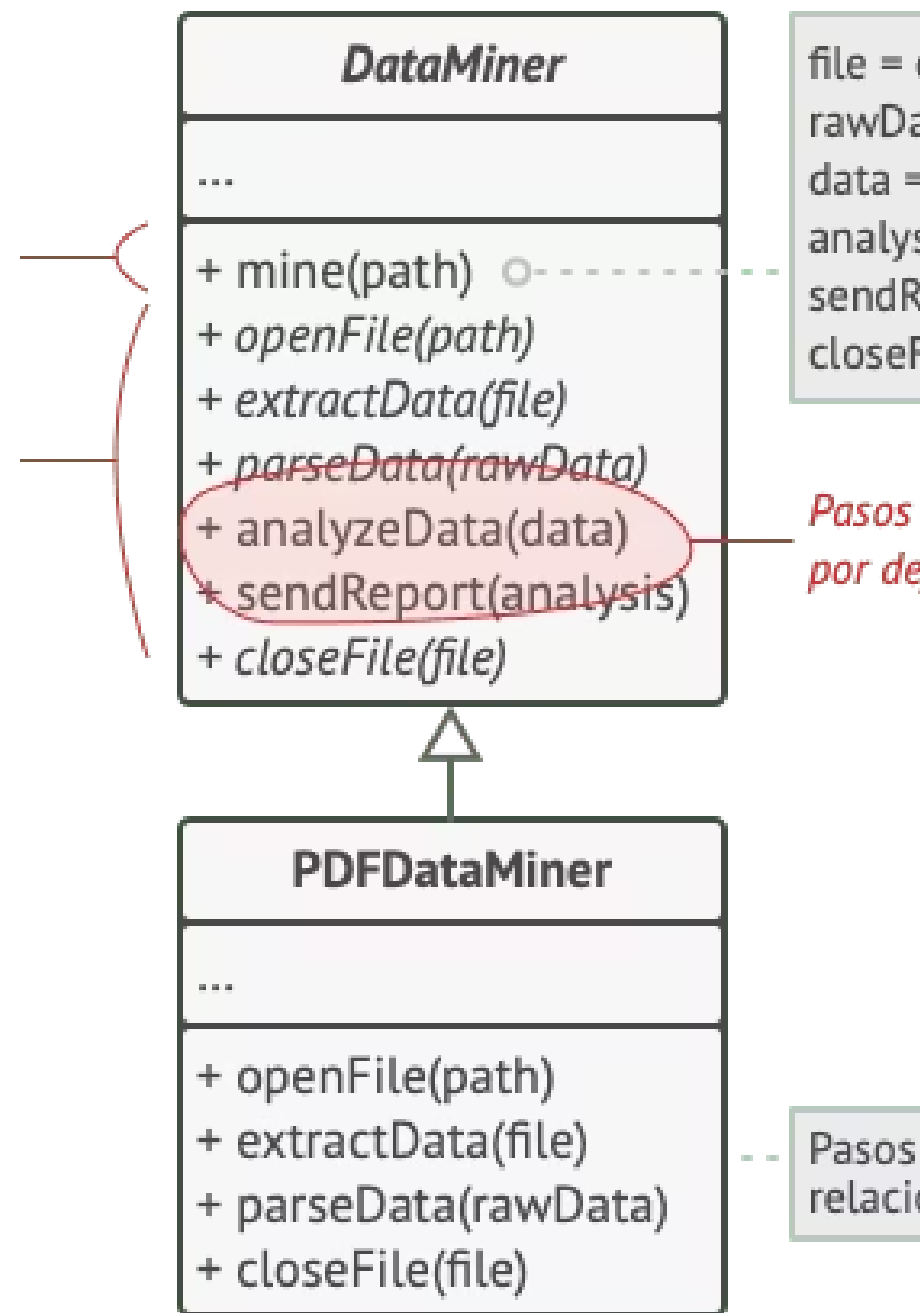
Método Plantilla

Orquesta las llamadas en el orden correcto

Solución Aplicada: Minería de Datos

Podemos crear una **clase base** para los tres algoritmos de análisis. Esta clase define un método plantilla consistente en una serie de llamadas a varios pasos de procesamiento de documentos.

El método plantilla divide el algoritmo en pasos, permitiendo a las subclasses sobrescribir estos pasos pero no el método en sí.



Ajustando la Implementación

Al principio, podemos declarar todos los pasos como **abstractos**, forzando a las subclases a proporcionar sus propias implementaciones. En nuestro caso, las subclases ya cuentan con todas las implementaciones necesarias, por lo que solo debemos **ajustar las firmas de los métodos** para que coincidan con los de la superclase.

Código diferente por formato

- Abrir/cerrar archivos
- Extraer información
- Analizar formato específico

No tiene sentido tocar estos métodos.

Código compatible

- Analizar datos sin procesar
- Generar informes
- Lógica de procesamiento común

Puede meterse en la clase base.

Tipos de Pasos en Template Method



Pasos Abstractos

Deben ser implementados por **todas** las subclases. Son obligatorios y definen el comportamiento específico de cada variante.



Pasos Opcionales

Ya tienen cierta **implementación por defecto**, pero aún así pueden sobrescribirse si es necesario para personalizar el comportamiento.

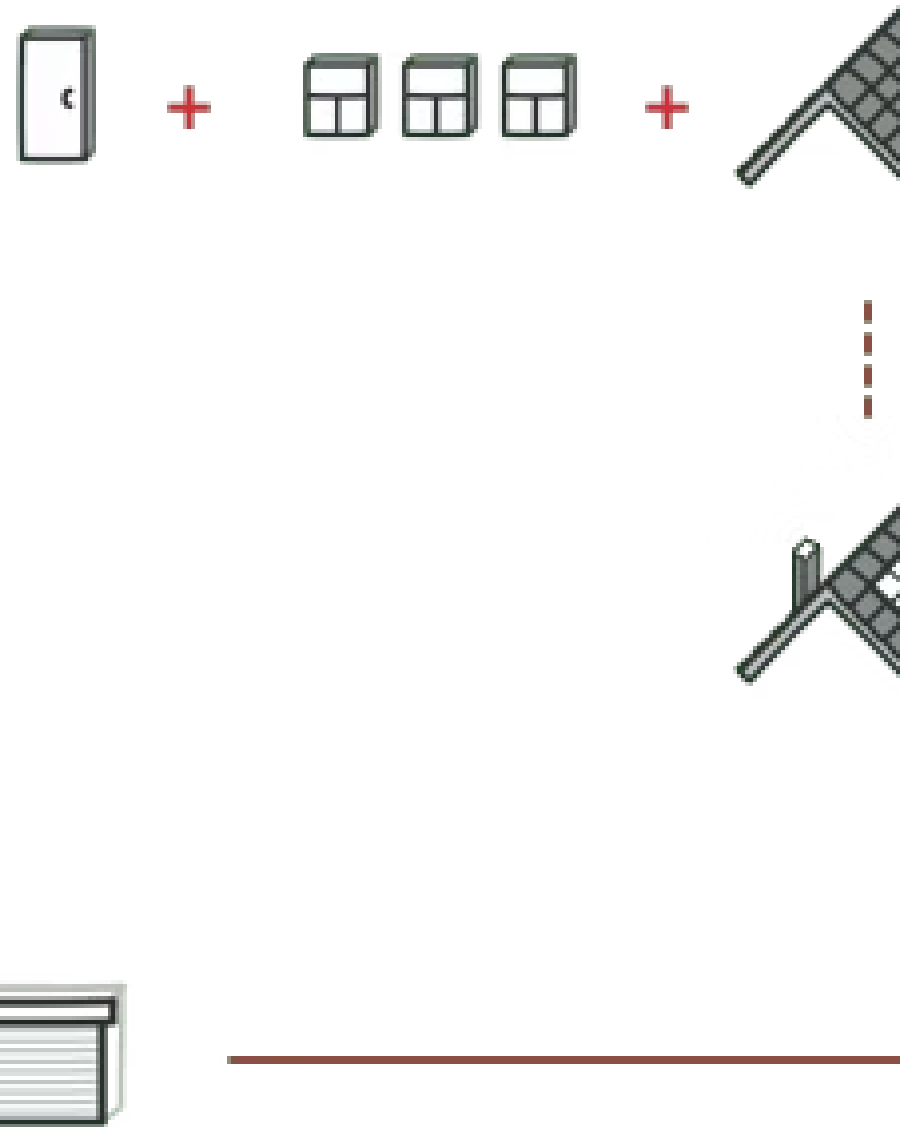


Ganchos (Hooks)

Paso opcional con un **cuerpo vacío**. El método plantilla funciona aunque no se sobrescriba. Se colocan antes y después de pasos cruciales como **puntos de extensión**.

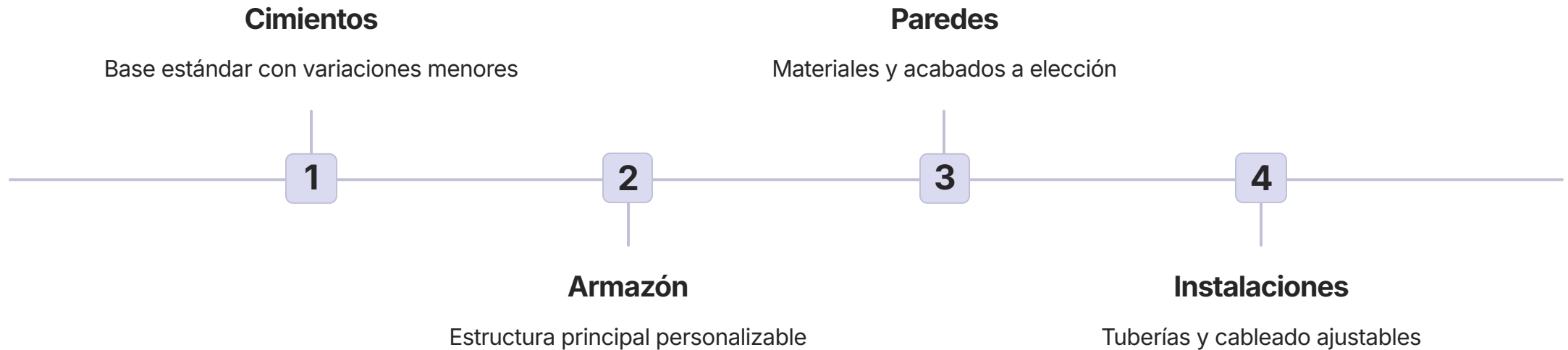
Analogía en el Mundo Real

Un plan arquitectónico típico puede alterarse ligeramente para que encaje mejor con las necesidades del cliente.

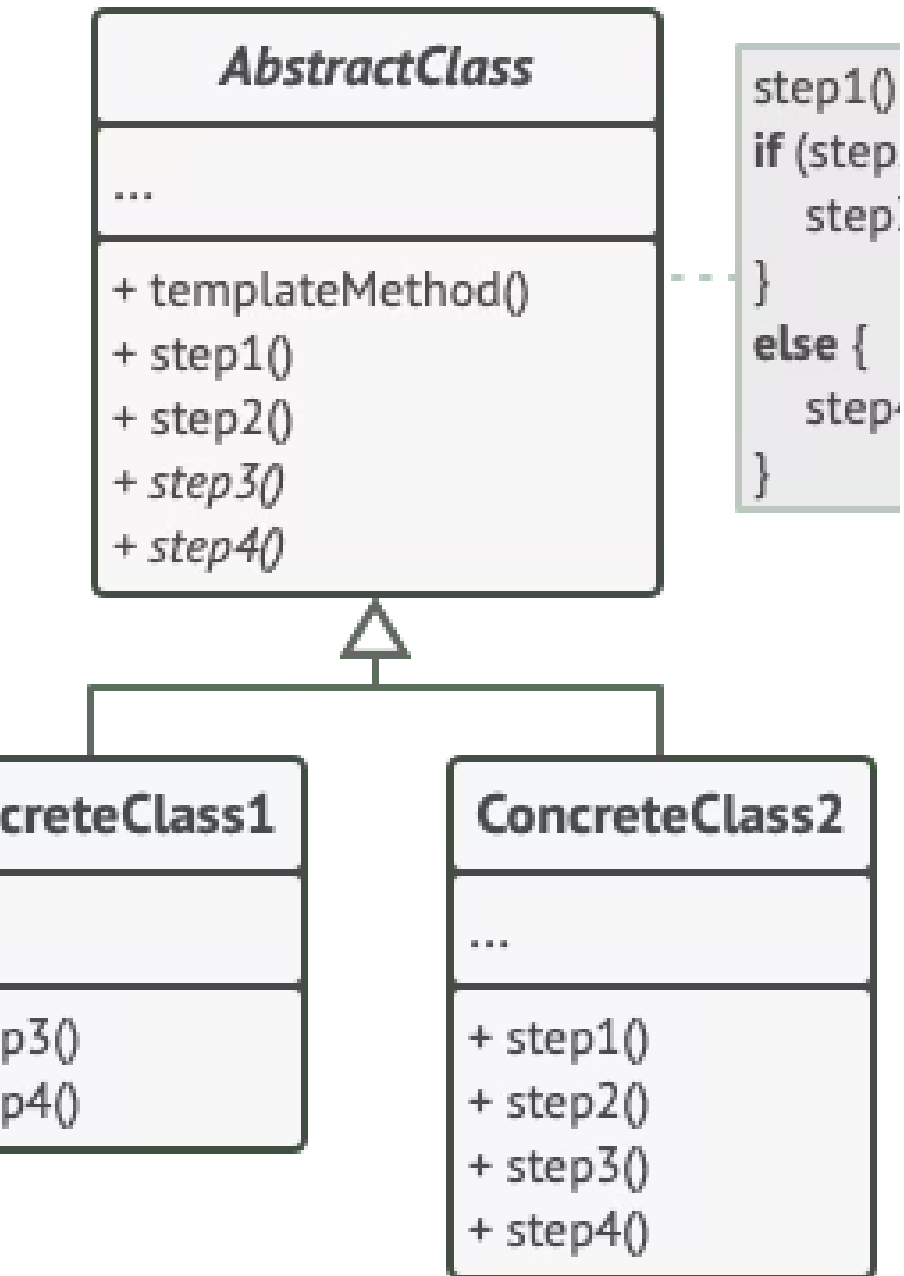


Construcción de Viviendas en Masa

El enfoque del método plantilla puede emplearse en la **construcción de viviendas en masa**. El plan arquitectónico para construir una casa estándar puede contener varios **puntos de extensión** que permitirán a un potencial propietario ajustar algunos detalles de la casa resultante.



Cada paso de la construcción puede cambiarse ligeramente para que la casa resultante sea **un poco diferente de las demás**.



Estructura del Patrón

Componentes de la Estructura

1

Clase Abstracta

Declara métodos que actúan como **pasos de un algoritmo**, así como el propio método plantilla que invoca estos métodos en un **orden específico**. Los pasos pueden declararse abstractos o contar con una implementación por defecto.

2

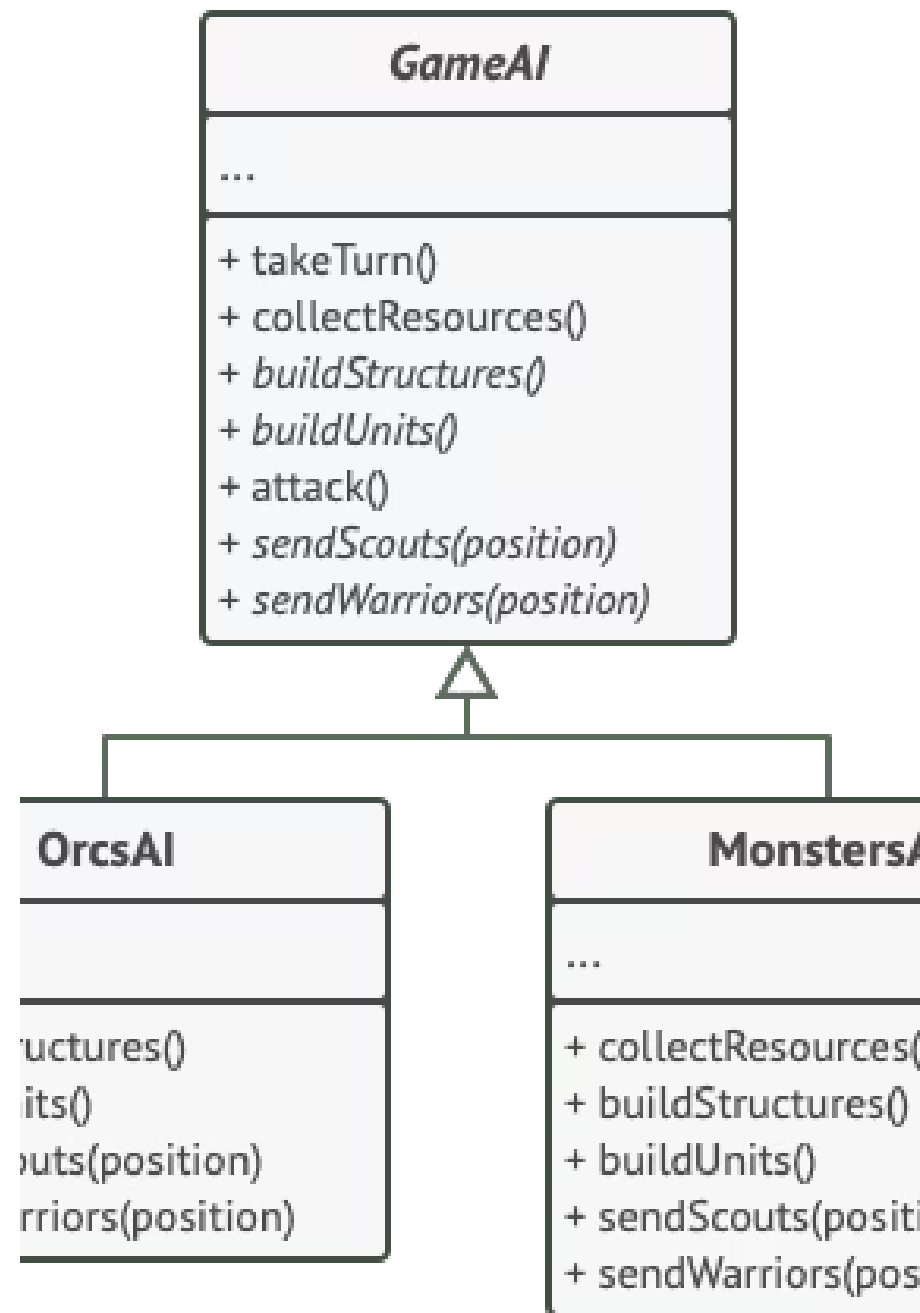
Clases Concretas

Pueden **sobrescribir todos los pasos**, pero **no el propio método plantilla**. Cada clase concreta proporciona su implementación específica de los pasos abstractos.

Pseudocódigo: Videojuego de Estrategia

En este ejemplo, el patrón Template Method proporciona un "**esqueleto**" para varias ramas de inteligencia artificial (IA) en un sencillo videojuego de estrategia.

Clases IA de un sencillo videojuego.



Razas del Juego y sus Variaciones

Todas las razas del juego tienen tipos de unidades y edificios casi iguales. Por lo tanto, puedes **reutilizar la misma estructura IA** para varias de ellas, a la vez que puedes sobrescribir algunos detalles.



Orcos

IA sobrescrita para ser **más agresivos**



Humanos

Actitud **más defensiva**



Monstruos

No pueden construir nada

Para añadir una nueva raza al juego habría que crear una **nueva subclase IA** y sobrescribir los métodos por defecto declarados en la clase IA base.

Pseudocódigo: Clase Abstracta GameAI

La clase abstracta define un método plantilla que contiene el esqueleto del algoritmo compuesto por llamadas a operaciones primitivas abstractas.

```
// La clase abstracta define un método plantilla que contiene un
// esqueleto de algún algoritmo compuesto por llamadas,
// normalmente a operaciones primitivas abstractas. Las
// subclasses concretas implementan estas operaciones, pero dejan
// el propio método plantilla intacto.
```

```
class GameAI is
```

```
    // El método plantilla define el esqueleto de un algoritmo.
```

```
    method turn() is
```

```
        collectResources()
```

```
        buildStructures()
```

```
        buildUnits()
```

```
        attack()
```

```
    // Algunos de los pasos se pueden implementar directamente
```

```
    // en una clase base.
```

```
    method collectResources() is
```

```
        foreach (s in this.builtStructures) do
```

```
            s.collect()
```

```
    // Y algunos de ellos pueden definirse como abstractos.
```

```
    abstract method buildStructures()
```

```
    abstract method buildUnits()
```

Pseudocódigo: Método `attack()` y Métodos Abstractos

Una clase puede tener **varios métodos plantilla**. El método `attack()` demuestra lógica con decisiones internas y delegación a métodos abstractos.

```
// Una clase puede tener varios métodos plantilla.  
method attack() is  
  enemy = closestEnemy()  
  if (enemy == null)  
    sendScouts(map.center)  
  else  
    sendWarriors(enemy.position)  
  
  abstract method sendScouts(position)  
  abstract method sendWarriors(position)
```

Pseudocódigo: Clase Concreta OrcsAI

Las clases concretas tienen que implementar **todas las operaciones abstractas** de la clase base, pero **no deben sobrescribir** el propio método plantilla.

```
// Las clases concretas tienen que implementar todas las
// operaciones abstractas de la clase base, pero no deben
// sobrescribir el propio método plantilla.
```

```
class OrcsAI extends GameAI is
```

```
method buildStructures() is
  if (there are some resources) then
    // Construye granjas, después cuarteles y después
    // fortaleza.
```

```
method buildUnits() is
  if (there are plenty of resources) then
    if (there are no scouts)
      // Crea peón y añádelo al grupo de exploradores.
    else
      // Crea soldado, añádelo al grupo de guerreros.
```

```
// ...
```

```
method sendScouts(position) is
  if (scouts.length > 0) then
    // Envía exploradores a posición.
```

```
method sendWarriors(position) is
  if (warriors.length > 5) then
    // Envía guerreros a posición.
```

Pseudocódigo: Clase Concreta MonstersAI

Las subclases también pueden **sobrescribir operaciones con una implementación por defecto**. Los monstruos anulan el comportamiento estándar dejando los métodos vacíos.

```
// Las subclases también pueden sobrescribir algunas operaciones
// con una implementación por defecto.
class MonstersAI extends GameAI is

    method collectResources() is
        // Los monstruos no recopilan recursos.

    method buildStructures() is
        // Los monstruos no construyen estructuras.

    method buildUnits() is
        // Los monstruos no construyen unidades.
```

- ❏ Observa cómo **MonstersAI** sobrescribe incluso `collectResources()`, que tenía implementación por defecto en la clase base, para anular completamente ese comportamiento.

Aplicabilidad

Extensión parcial de algoritmos

Utiliza el patrón cuando quieras permitir a tus clientes que extiendan **únicamente pasos particulares** de un algoritmo, pero no todo el algoritmo o su estructura. Te permite convertir un algoritmo monolítico en una serie de **pasos individuales** que se pueden extender fácilmente con subclases.

Algoritmos casi idénticos

Utiliza el patrón cuando tengas **muchas clases que contengan algoritmos casi idénticos**, pero con algunas diferencias mínimas. Puedes elevar los pasos con implementaciones similares a una superclase, **eliminando la duplicación del código**. El código que varía puede permanecer en las subclases.

Cómo Implementarlo

1 Analizar el algoritmo

Divídelo en pasos. Considera qué pasos son **comunes** a todas las subclases y cuáles siempre serán **únicos**.

2 Crear la clase base abstracta

Declara el método plantilla y un grupo de **métodos abstractos** que representen los pasos. Considera declarar el método plantilla como **final**.

3 Implementaciones por defecto

Algunos pasos pueden tener una **implementación por defecto**. Las subclases no tienen que implementar esos métodos.

4 Añadir ganchos

Piensa en añadir **hooks** entre los pasos cruciales del algoritmo como puntos de extensión adicionales.

5 Crear subclases concretas

Para cada variación, crea una nueva subclase que implemente todos los pasos abstractos y **sobrescriba los opcionales** si es necesario.

Pros y Contras

✅ Ventajas

- Puedes permitir a los clientes que sobrescriban **tan solo ciertas partes** de un algoritmo grande, para que les afecten menos los cambios en otras partes del algoritmo.
- Puedes colocar el **código duplicado** dentro de una superclase.

❌ Desventajas

- Algunos clientes pueden verse **limitados** por el esqueleto proporcionado de un algoritmo.
- Puede que violes el **principio de sustitución de Liskov** suprimiendo una implementación por defecto de un paso a través de una subclase.
- Los métodos plantilla tienden a ser **más difíciles de mantener** cuantos más pasos tengan.

Relaciones con Otros Patrones



Factory Method

[Factory Method](#) es una **especialización** del Template Method. Al mismo tiempo, un Factory Method puede servir como **paso** de un gran Template Method.



Strategy

[Template Method](#) se basa en la **herencia**: altera partes de un algoritmo extendiendo en subclases. [Strategy](#) se basa en la **composición**: altera partes del comportamiento suministrando distintas estrategias.

- ❏ **Template Method** trabaja al nivel de la **clase**, por lo que es estático. **Strategy** trabaja al nivel del **objeto**, permitiéndote cambiar los comportamientos durante el tiempo de ejecución.

Resumen: Template Method



Esqueleto

Define la estructura del algoritmo en la superclase



Personalización

Las subclasses sobrescriben pasos sin alterar la estructura



Reutilización

Elimina código duplicado elevándolo a la clase base

El patrón Template Method es fundamental para diseñar algoritmos extensibles y mantener el principio **DRY** (Don't Repeat Yourself) en tu código orientado a objetos.

